



Automated assembly quality inspection by deep learning with 2D and 3D synthetic CAD data

Xiaomeng Zhu^{1,3} · Pär Mårtensson¹ · Lars Hanson² · Mårten Björkman³ · Atsuto Maki³

Received: 23 June 2023 / Accepted: 14 March 2024 / Published online: 18 April 2024
© The Author(s) 2024

Abstract

In the manufacturing industry, automatic quality inspections can lead to improved product quality and productivity. Deep learning-based computer vision technologies, with their superior performance in many applications, can be a possible solution for automatic quality inspections. However, collecting a large amount of annotated training data for deep learning is expensive and time-consuming, especially for processes involving various products and human activities such as assembly. To address this challenge, we propose a method for automated assembly quality inspection using synthetic data generated from computer-aided design (CAD) models. The method involves two steps: automatic data generation and model implementation. In the first step, we generate synthetic data in two formats: two-dimensional (2D) images and three-dimensional (3D) point clouds. In the second step, we apply different state-of-the-art deep learning approaches to the data for quality inspection, including unsupervised domain adaptation, i.e., a method of adapting models across different data distributions, and transfer learning, which transfers knowledge between related tasks. We evaluate the methods in a case study of pedal car front-wheel assembly quality inspection to identify the possible optimal approach for assembly quality inspection. Our results show that the method using Transfer Learning on 2D synthetic images achieves superior performance compared with others. Specifically, it attained 95% accuracy through fine-tuning with only five annotated real images per class. With promising results, our method may be suggested for other similar quality inspection use cases. By utilizing synthetic CAD data, our method reduces the need for manual data collection and annotation. Furthermore, our method performs well on test data with different backgrounds, making it suitable for different manufacturing environments.

Keywords Assembly quality inspection · Synthetic data · Computer vision · Unsupervised domain adaptation · Transfer learning · Point cloud

Introduction

In the next generation of industrial system, Industry 4.0, digitalization and automation are the core of the manufacturing industry (Oesterreich and Teuteberg 2016). With a focus on automation, automated quality inspection can effectively enhance product quality and productivity, making it a critical consideration for companies seeking to meet customer expectations. However, automating the quality inspection of

complex manufacturing processes such as assembly, which involves various products and operator activities, can be challenging. The trend from mass production to mass customization is expected to further increase the diversity and complexity of assembly and automatic assembly quality inspection (Michalos et al. 2010)

Deep learning, as the cutting-edge technology in computer vision, has proven its effectiveness in various industry applications and may offer a potential solution for automatic assembly quality inspection (Wang et al. 2018; Mazzetto et al. 2020). However, deep learning models usually assume that the training and testing data are drawn from the same distribution, i.e., supervised learning (SL), making the models rely on a large number of annotated training samples (Li et al. 2020). Collecting and annotating training data for assembly

✉ Xiaomeng Zhu
xiazhu@kth.se

¹ Scania CV AB (publ), Södertälje 151 87, Sweden

² University of Skövde, Skövde 541 28, Sweden

³ KTH Royal Institute of Technology, Stockholm 100 44, Sweden

processes is a time-consuming and labor-intensive task that requires substantial manual effort (Maschler et al. 2020).

This challenge might be addressed using synthetic data. Synthetic data can be an essential approach to overcome the problem of insufficient data by either producing artificial data from scratch or using advanced data manipulation technologies to produce novel and diverse training examples (Nikolenko 2021). In the manufacturing industry, computer-aided design (CAD) models are widely used to create elaborate computerized models of objects before they are physically produced (Tovey 1989). CAD models are also used in virtual simulation, planning, and training (Leu et al. 2013). Therefore, many researchers have attempted to use synthetic data generated from CAD models to solve the data problem in the manufacturing industry (Cohen et al. 2020; Dekhtiar et al. 2018; Wong et al. 2019; Horváth et al. 2022). As synthetic CAD data are generated with annotation, their use can reduce the workload of manual data collection and annotation. However, a challenge associated with synthetic CAD data lies in the domain gap between the CAD data and real data, as they are from different distributions. This gap may hinder the effective generalization of models trained on CAD data to real-world scenarios. The reason is that synthetic data can lack the complexity of real-world conditions, such as environmental noise, and may not precisely simulate the physics of reflections and textures.

A widely used method to map the domain gap between data from different distributions is transfer learning (TL). TL focuses on transferring knowledge between different domains or tasks to improve the performance of the model (Zhuang et al. 2020). Fine-tuning is a common approach in TL, where a model is trained with data from a source domain (i.e., synthetic data) and then fine-tuned with the data from a target domain (i.e., real data). Another approach, domain adaptation (DA), is the research associated with TL and aims to train a model on a label-rich dataset (i.e., source domain) and a label-poor dataset (i.e., target domain) (Qin et al. 2019). The trained model is expected to perform well in the target domain. State-of-the-art research on TL and DA can be found in (Li, Li, Luo, Wang, et al., 2020; Zhuang, Qi, Duan, Xi, Zhu, Zhu, Xiong, and He, 2020).

To the best of our knowledge, while existing research in industrial applications with synthetic data has primarily focused on generating 2D images and applying TL methods (Cohen et al. 2020; Dekhtiar et al. 2018; Wong et al. 2019; Horváth et al. 2022), there remains a gap in studies that adopt both TL and DA methods and compare their performance. Additionally, no research has yet generated the synthetic data in both 2D and 3D formats for a comparative analysis to identify which format is more effective for industrial applications.

This paper proposes an automatic assembly quality inspection method utilizing synthetic data generated from CAD models. The method has two steps: 1) automatic data

generation and 2) model implementation. The first step involves generating synthetic data from CAD models, while the second step implements state-of-the-art deep learning models to use the synthetic data for assembly quality inspection. To provide initial insights into the more effective method for assembly quality inspection, we explore the use of synthetic data in both 2D and 3D scopes, where the data are represented in RGB image format in the 2D scope and point cloud format in the 3D scope. We then implemented and compared the TL and DA deep learning models in both scopes. The effectiveness of the proposed method was evaluated through an industrial case study of pedal car front-wheel assembly quality inspection.

The contributions of this paper can be summarized as follows:

- A method for assembly quality inspection using synthetic data generated from CAD models is proposed, offering a time-efficient and cost-saving solution to reduce manual data collection and annotation.
- Synthetic data in both 2D and 3D scopes are generated and compared. Through comprehensive evaluation, the findings indicate that the 2D scope might exhibit greater suitability for industrial applications.
- State-of-the-art transfer learning (TL) and domain adaptation (DA) models are applied to the synthetic data in an industrial case study of pedal car front-wheel assembly quality inspection. The results suggest the TL models may be more suitable for assembly quality inspection.

The rest part of this paper is organized as follows. In “Related work” section, we explore previous studies and theoretical frameworks that inform our approach. Section “Industrial case study” describes the specific case we investigated, demonstrating the practical application of our method. Section “Method and implementation” details the techniques and processes we used in our study. Section “Experiments” outlines our experimental procedures and setup. Section “Experimental results and discussion” presents an analysis of the outcomes of these experiments. Our paper concludes with “Conclusion” section, where we summarize the key findings, discuss the limitations of our study, and suggest directions for future research.

Related work

Deep learning in quality inspection with 2D images

Deep learning, as a popular approach to leverage the potential of data, has been proven successful for different applications in manufacturing, including quality inspection (Wang et al. 2018; Krauß et al. 2020). Among various deep learning mod-

els, object detection has attracted much attention in industry quality inspection. The survey by (Ahmad and Rahimi, 2022) has summarized different deep learning models for object detection in manufacturing processes.

There are two types of object detection models: two-stage detectors, such as Faster R-CNN (FRCNN) (Ren et al. 2015) and DetectoRS (Qiao et al. 2021), and one-stage detectors, including SSD (Liu et al. 2016), Yolov4 (Bochkovskiy et al. 2020), and Swin Transformer (Liu et al. 2021). Two-stage detectors typically offer higher object recognition precision, while one-stage detectors tend to have faster inference speeds (Jiao et al. 2019). Among different object detection models, despite being developed in 2016, FRCNN is still considered the state-of-the-art baseline model and has been widely employed in smart manufacturing (Ahmad and Rahimi 2022). In addition, it has achieved good performance in the small object detection task (Nguyen et al. 2020) and scored highly on some image benchmark datasets, such as PASCAL VOC (Everingham et al. 2010) according to the latest object detection models survey (Zaidi et al. 2022).

Deep learning in quality inspection with 3D point cloud

Point cloud data are becoming increasingly important in quality inspection due to their ability to provide richer information and their reduced sensitivity to background and illumination compared with RGB images (Zhu et al. 2021). There are two popular methods of utilizing point clouds for manufacturing quality inspection: 1) analysing the presence and pattern of point clouds (Dastoorian et al. 2018; Huang and Kovacevic 2011); 2) comparing point clouds with a preferred CAD surface to detect deviations through point cloud registration (Zhang et al. 2008). Despite these approaches, as far as the authors are aware, little research has adopted deep learning models on point clouds for manufacturing quality inspection.

Several surveys (Guo et al. 2020; Liu et al. 2019) have summarized the different deep learning methods applied to point clouds and their applications. PointNet (Qi et al. 2017a) was the first deep learning model developed to work directly with point clouds. Building upon PointNet, PointNet++ (Qi et al. 2017b) focuses on both global and local geometric information of point clouds and has achieved outstanding results on benchmark datasets such as ModelNet (Wu et al. 2015) and ScanNet (Dai et al. 2017). Thus, although PointNet++ was introduced in 2017, it remains the state-of-the-art baseline for point cloud data.

Deep learning in quality inspection with limited training data

One of the most significant challenges in deploying deep learning models in industry is the availability of sufficient training data (Munappy et al. 2022). To address this issue, researchers have developed various techniques to overcome data scarcity. For instance, (Li, Zhang, Ding, and Sun, 2020) employed data augmentation to increase the training data for intelligent rotating machinery fault inspection, while (Krüger, Lehr, Schlueter, and Bischoff, 2019) focused on inherent features and (Maschler, Kamm, Jazdi, and Weyrich, 2020) used incremental learning for industry part recognition. Synthetic data generated from CAD models have also been used to expand training datasets for deep learning in various industrial applications, as described in Cohen et al. (2020); Dekhtiar et al. (2018); Wong et al. (2019); Horváth et al. (2022). However, most of this research has predominantly focused on 2D applications. To the best knowledge of the authors, no research has yet generated 3D synthetic data for quality inspection. Moreover, while most research employs SL and TL techniques, the implementation of domain adaptation methods on synthetic data remains unexplored.

Synthetic data

Synthetic data are a widely recognized approach for addressing the challenge of data limitations in deep learning. In the book *Synthetic Data for Deep Learning* (Nikolenko 2021), the different methods for generating and utilizing synthetic data in deep learning are summarized. Synthetic data can be produced either from scratch or by employing advanced data manipulation processes to enhance the variability of existing data (Nikolenko 2021). In a survey of image synthesis methods (Tsirikoglou et al. 2020), the approaches to generating synthetic data were divided into four categories: content generation, which simulates virtual objects and environments; image synthesis, which renders images by simulating light sources scattered at the surface and is commonly used in the gaming industry; learning-based image synthesis, which leverages deep learning-based generative models such as generative adversarial networks (GANs) and diffusion networks to create data; and data augmentation, which produces new data by modifying and varying existing data.

In the generation of synthetic data, two primary strategies are commonly employed. The first involves creating photo-realistic data that closely resemble real-world scenarios, thereby allowing the synthetic data to accurately represent real data. This enables models trained on such synthetic data to apply their learning to actual environments effectively (Horváth et al. 2022). The second strategy, domain randomization, introduces a wide range of variations into the synthetic data. This diversity aims to improve the gen-

eralization capabilities of the model and enable it to learn domain-independent representation. Therefore, the model may not overfit to specific domain characteristics and consider the real data as merely another variation of the synthetic data (Tobin et al. 2017).

Based on these strategies, numerous benchmark synthetic datasets have been created to study how to narrow the domain gap between synthetic data and real data. One such dataset is SIM 10K (Johnson-Roberson et al. 2016), which consists of synthetic street view images rendered from the video game Grand Theft Auto V (GTA5). It is commonly studied in conjunction with the Cityscapes Dataset (Cordts et al. 2016), which contains meta-street images collected via GPS, in simulation-to-real (Sim-to-Real) research in 2D scope. In 3D scope, the ModelNet dataset (Wu et al. 2015), which includes point clouds of furniture generated from CAD models, and the ScanNet dataset (Dai et al. 2017), which contains point clouds of furniture captured with 3D sensors, are studied together.

Deep learning models for data from different distributions

Research on synthetic data (de Melo et al. 2021; Nikolenko 2021; Seib et al. 2020; Tsirikoglou et al. 2020) has summarized varied deep learning methods that deal with the distribution gap between synthetic data and real data.

Transfer learning (TL) is a method that transfers the trained weights from the synthetic distribution to the real distribution. Typically, the network is first trained on the data from one distribution, i.e., the source domain. Then the trained network is used as a backbone to initiate the weight of a new network and trained on data from the other distributions, i.e., the target domain (Seib et al. 2020). This process is called fine-tuning. The backbone is considered a feature extractor. It learns the low-level features that differ less across domains from the source domain (Seib et al. 2020), then shifts this knowledge to the target domain to improve the model performance on the target domain. In the context of transfer learning, few-shot transfer learning refers to fine-tuning the model with only a few samples from the target domain (Wang et al. 2020). This approach could prove advantageous in assembly quality inspection as it reduces the efforts of manual data collection.

Domain adaptation (DA) is the research associated with TL. It involves training a robust deep learning model on a label-rich dataset from one distribution, i.e., the source domain, and a label-poor dataset from another distribution, i.e., the target domain (Miller 2019). The model is expected to perform well in the target domain. Among domain adaptation, unsupervised domain adaptation (UDA) does not require any annotation from the target domain (Zhu et al.

2021), making it appealing to assembly quality inspection since it can reduce manual annotation efforts.

Since object detection networks are of interest for industrial quality inspection with 2D images, unsupervised domain adaptive object detection networks may be employed on 2D synthetic data for assembly quality inspection. The survey by (Li, Li, Luo, Wang, et al., 2020) has summarized the state-of-the-art domain adaptive object detection network and divided them into four categories: discrepancy-based network, adversarial-based network, reconstruction-based network, and hybrid network (Zhu et al. 2022). Adversarial-based networks have shown promising results in Sim-to-Real research. These networks bridge the domain gap using a discriminator that aims to deceive it into being unable to identify the discrepancy between the source and target domains. The survey by (Li, Li, Luo, Wang, et al., 2020) compared the performance of state-of-the-art unsupervised domain adaptive object detection models on the SIM 10K and Cityscapes datasets. The strong-weak distribution alignment (SWDA) model (Saito et al. 2019) achieved the best performance on the datasets.

Most domain adaptation networks that focus on 2D images consider only global feature alignment, which refers to high-level features such as object categories and scene types that capture the overall shape and structure of an object or scene. Examples of such networks include the maximum classifier discrepancy (MCD) network and domain-adversarial neural network (DANN) (Ganin et al. 2016). However, in 2019, a 3D domain adaptation network for point cloud data called PointDAN (Qin et al. 2019) was proposed. Unlike the 2D networks, PointDAN jointly aligns global and local features, which refer to low-level features such as edges and corners that capture specific details or patterns within an object or scene. This approach has been shown to improve the performance of domain adaptation on point cloud data, as demonstrated on the ModelNet and ScanNet datasets (Qin et al. 2019).

Industrial case study

In this paper, we have chosen a pedal car front-wheel assembly as a case study to examine assembly quality inspection. Pedal cars, which are simplified versions of vehicles, are widely used for assembly training and research in companies and universities. For example, Scania uses them to introduce new employees to the assembly process (Samir et al. 2018). They are also used for research in various domains, such as cyber-physical production systems (Samir et al. 2018), manual assembly (Brolin et al. 2017), and lean production (de Vin et al. 2018).



Fig. 1 A pedal car during assembly

Figure 1 shows a pedal car during assembly. Four various assembly approaches can be applied while assembling the front wheel of the pedal car, as shown in Fig. 2:

- (a) Class 1, assembled on the correct side of the wheel with a screw
- (b) Class 2, assembled on the correct side of the wheel without a screw
- (c) Class 3, assembled on the wrong side of the wheel with a screw
- (d) Class 4, assembled on the wrong side of the wheel without a screw

In this case study, we aim to train a model to classify images of these four categories to enable automatic quality inspection. This case study was chosen since it may represent the main challenges associated with deploying deep learning techniques for assembly quality inspection. One challenge is the high time and labor cost of manually collecting and annotating large amounts of training data for supervised learning. In the use case, during the manual assembly process, movements of the front wheel and front steering shaft can guide the front wheel to varied assembly end positions, while environmental factors such as illumination and background can further increase variability, thereby complicating the manual data collection and annotation process.

Another challenge is the fine-grained level classification. Unlike coarse-grained classification, which groups items with significant differences, such as wheels, steering wheels, pedals, and other industrial parts, fine-grained classification involves categories with high similarities, such as wheels with or without screws, and wheels with different rims (Chou et al. 2022). In this case study, the differences among the four classes are insignificant, making it challenging to distinguish

between them. For instance, the dark color of the wheel rim and background can make it difficult to discern the assembled sides of the rim. Similarly, the screw is small and has the same color as its hole, making it challenging to determine whether the screw is correctly assembled. Hence, this case study requires fine-grained classification, which may be more complex than the coarse-grained level analysis used in most synthetic data research.

In summary, this case study is adequately challenging for automated assembly quality inspection. Therefore, the successful implementation of the proposed methodology could potentially provide valuable insights and guidelines for other automated assembly inspection systems.

Method and implementation

A method with two steps is proposed in this paper. The method is followed in both 2D and 3D scopes. The 2D method is summarized in Fig. 3, and the 3D method is outlined in Fig. 4.

Step 1: Generate synthetic data from CAD models as the source domain data, named CAD data, and capture data with cameras from a real manufacturing environment as the target domain data, named CAM data. In this study, we generate and capture data in both 2D and 3D scopes.

In the 2D scope, RGB images are used as the data. To ensure that the generated CAD data can simulate different assembly approaches, we first set the CAD models in simulation software with virtual cameras and environments. Synthetic images are then rendered from the virtual camera using the domain randomization technique proposed by (Tobin, Fong, Ray, Schneider, Zaremba, and Abbeel, 2017). This approach generates synthetic data with sufficient variations to enable the network to view real-world data as just another variation. Previous studies have demonstrated using physics-based rendering and domain randomization to generate 2D synthetic images for industrial parts recognition (Eversberg and Lambrecht 2021; Horváth et al. 2022; Zhu et al. 2023). Following these ideas, we generated synthetic images with random CAD model sizes, positions, rotations, camera views, illuminations, backgrounds, and noise (Zhu et al. 2022), as shown in Fig. 3a.

The 2D CAM data are captured using a 2D camera in real industrial environments, as shown in Fig. 3b.

In 3D scope, point clouds are used as the data because they consist of points with 3D coordinates, and they are the most straightforward representation to preserve 3D spatial information (Qin et al. 2019). To generate CAD data that simulate various assembly approaches, we utilized simulation software to create 3D meshes with different assembly end positions. These meshes were then converted into point clouds, and irrelevant parts were cropped out to focus on the

Fig. 2 A pedal car front wheel assembly

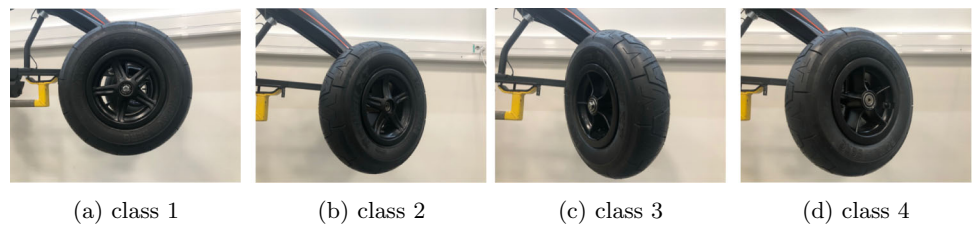
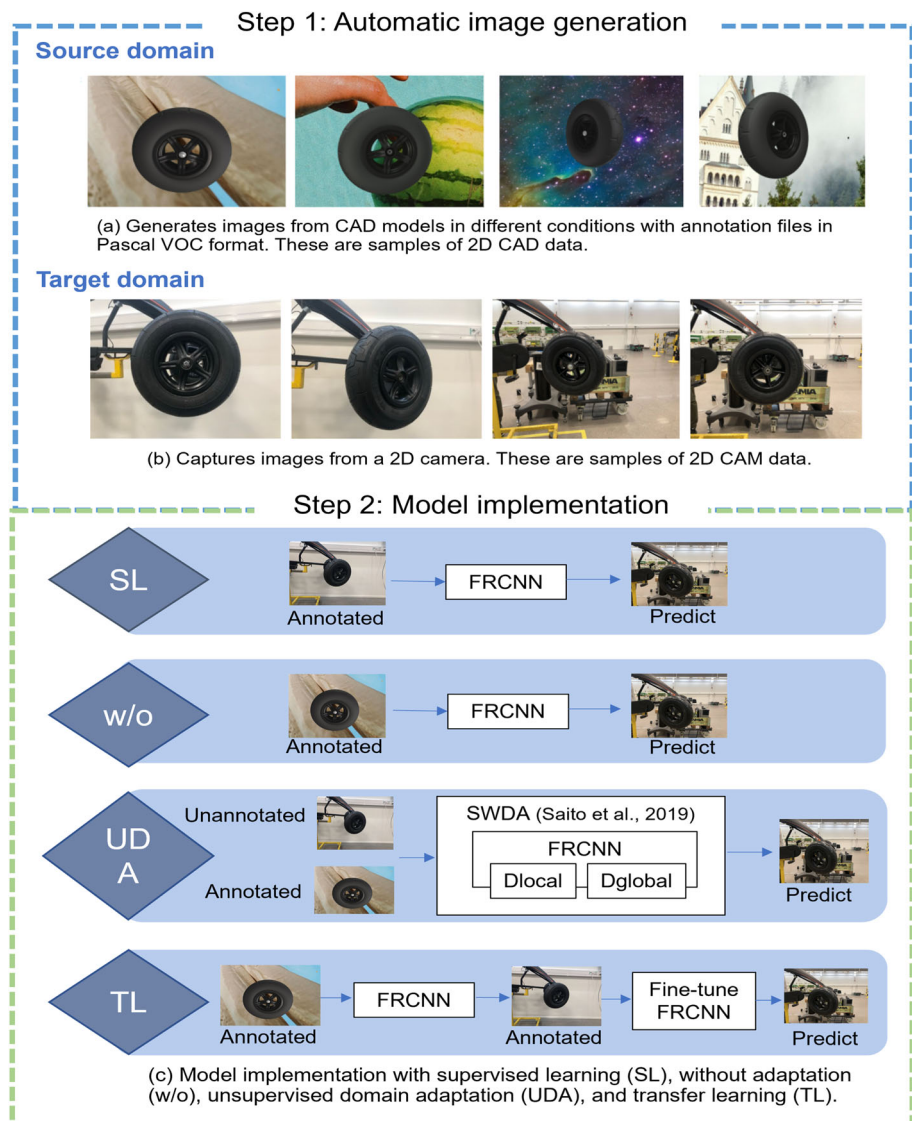


Fig. 3 The method in 2D scope

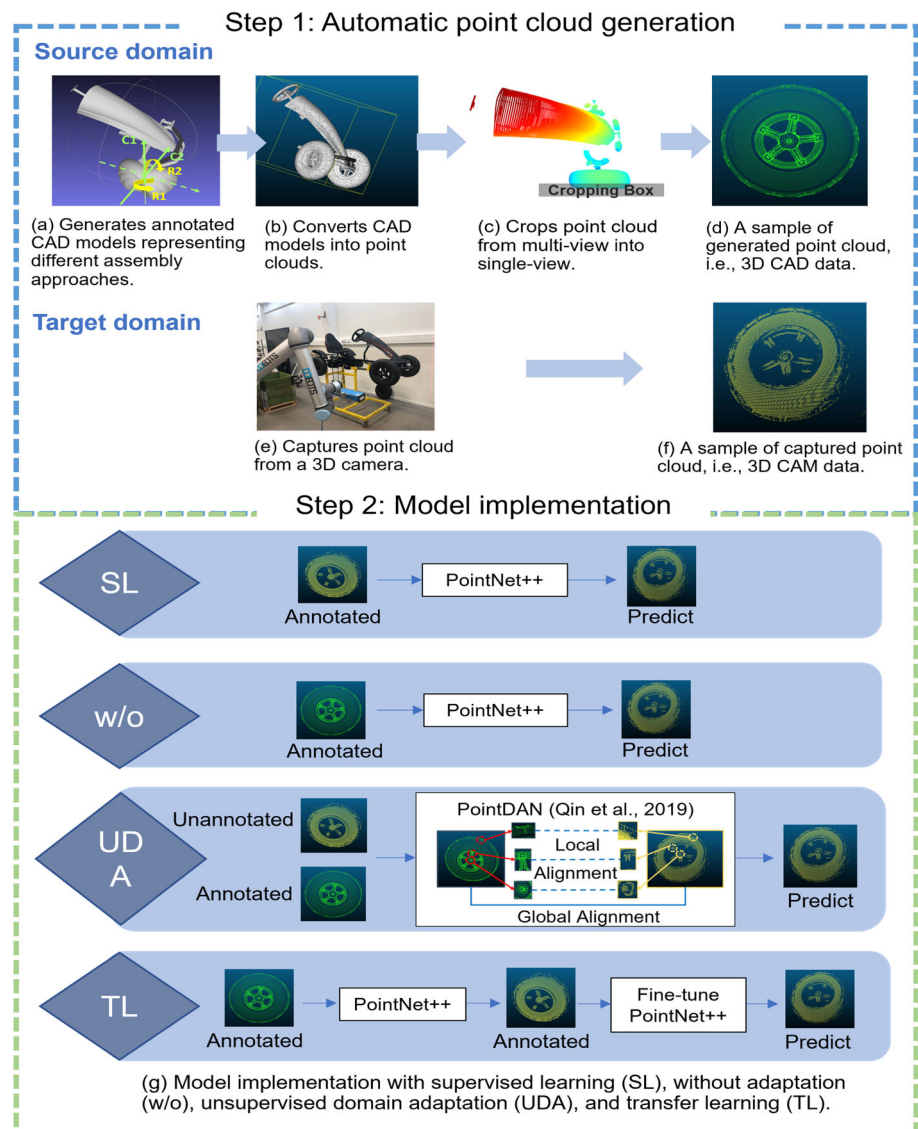


part requiring inspection. The process is presented in Fig. 4a–c.

The 3D CAM data are captured using a 3D camera. To capture high-resolution point clouds, a scannable 3D laser triangulation camera is required. We mounted the 3D camera on a robot to scan the front wheel of the pedal car, as shown in Fig. 4e.

The specifics of implementing the data generation process in the case study are outlined in “Automatic data generation” section.

Step 2: Employ the state-of-the-art deep learning models on the dataset generated from Step 1 for assembly quality inspection and compare their performance. In this study, we have chosen to implement UDA and TL methods as they are specifically designed to handle data from different distributions. Although both methods require both CAD data and CAM data for training, the UDA method does not require manual annotations, while the TL method only requires a small amount of annotated CAM data. Therefore, these meth-

Fig. 4 The method in 3D scope

ods can save manual data collection and annotation work compared with the SL method.

In the UDA experiment, the model is trained with annotated CAD data and unannotated CAM data. In the TL experiment, the model is trained first with a large amount of annotated CAD data and then fine-tuned with a small amount of annotated CAM data. The performance of the UDA and TL methods will be compared with the SL method and the method without any adaptation (w/o).

In the SL method, a model is trained only with annotated CAM data. Since the training and testing data are from the same distribution, the SL method does not need to bridge the domain gap. This is a commonly used deep learning method for automated assembly quality inspection, only relying on real data. Conversely, in the w/o method, a model is trained only with annotated CAD data, so the training and testing data are from different distributions, but the model is without any

adaptation. Therefore, both the SL and w/o methods serve as baselines for the UDA and TL method. To compare their performance, all methods are applied to the same case study and tested on the same CAM data.

In the 2D scope, as shown in Fig. 3b, since the RGB images captured contain background elements unrelated to the target component, object detection models are employed to inspect the assembly quality. On the other hand, in the 3D scope, as presented in Fig. 4f, since the point clouds of the target component contain no extraneous background information, classification models are utilized to perform assembly quality inspection.

Further details regarding the selection of models for various methods applied in the case study are described in “Model implementation” section.

Automatic data generation

Step 1 of the method is to automatically generate annotated synthetic CAD data, which can represent different assembly approaches. In the case study of the pedal car front-wheel assembly, the movement of the steering shaft and the rotation of the wheel can make the front wheel end in uncertain positions after manual assembly. Therefore, the generated CAD data should be able to represent all possible end positions of the front wheel.

To achieve this, in 2D scope, four CAD models were prepared, each representing a distinct assembly category. Each CAD model contains only the wheel and the screw. To provide a reasonably realistic appearance, they were given single-color textures that approximate the real objects. These models were then imported into a simulation software built on Unity (Haas 2014), where one virtual camera and six distinct lights were set up to focus on the CAD models. The camera angle was set to a maximum of 120 degrees since the inspection of the wheel only requires a view from the front. To achieve domain randomization, the CAD data were generated by rendering CAD models from various camera angles and lighting intensities, with each model being randomly rotated, scaled, and placed. In addition, we incorporated randomly selected backgrounds from the Unsplash dataset (Unsplash 2020) along with various post-processing techniques such as random color tints, blurs, and noise into the CAD data (Zhu et al. 2022; Zhu et al. 2023). We followed the rendering parameters described by (Rangarajan, Gupta, Andreas, Breitenfeld, Schulz, Ling, Kammerlocher, and Baier, 2022).

In this study, for the source domain, 1200 images were generated per class, i.e., $4 \times 1200 = 4800$ images were generated in total as 2D CAD data. Samples of the 2D CAD data are shown in Fig. 3a. All images were generated with XML annotation files in Pascal VOC format (Everingham et al. 2010).

For the target domain, 380 images were captured by an iPhone X with different backgrounds and illuminations in a real-life manufacturing scenario as 2D CAM data. Samples of 2D CAM data are shown in Fig. 3b. All images in the 2D CAD and CAM datasets share the same resolution, specifically, 4032×3024 pixels.

In 3D scope, four CAD models representing different assembly categories were also prepared. The texture was not required since point clouds do not contain texture information. These CAD models might contain more pedal car parts besides the wheel and screw, as seen in Fig. 4a. Three tools were then used to generate the CAD data. The first tool was the Industrial Path Solutions (IPS) simulation software, where CAD models were imported and modified through Lua language to simulate various assembly end positions (Zhu et al. 2021). In this study, two modifications, R1 and R2, were applied simultaneously to the front-wheel CAD model. As

shown in Fig. 4a, R1 simulated the movement of the steering shaft by rotating the wheel around axis C1 for 2 degrees per time, 30 times, and R2 simulated the movement of the wheel by rotating the wheel around axis C2 for 9 degrees per time, 40 times. Altogether, 40 (R1 modification) $\times$ 30 (R2 modification) $= 1200$, $\times$ 4 (CAD model classes) $= 4800$ CAD models were generated from IPS.

The output files from IPS were 3D meshes in .wrl format. The second tool, MeshLab, a 3D processing software, was then used to batch convert the 3D meshes into point clouds in .npy format, as shown in Fig. 4b.

However, the generated point clouds still contain unnecessary parts besides the front wheel and screw and are represented in multi-view mode, while point clouds captured by 3D cameras are represented in single view. To address this, the third tool, Open3D, an open-source Python library, is used to crop the generated point clouds into a single view and filter out the unrelated parts (Zhu et al. 2021). Figure 4c shows the point clouds cropped by a customized cropping box. The length and width of the box are decided by the capturing range of a 3D camera, which simulates how a 3D camera acquires data. The cropping box needs to be manually created once. Then its coordinates are remembered and automatically applied to all the selected point cloud files. A sample of the point cloud after cropping is shown in Fig. 4d.

For the target domain, 611 point clouds are captured by a 3D laser triangulation camera SICK TrispectorP1000. The camera setting is shown in Fig. 4e, and a sample of 3D CAM data is presented in Fig. 4f.

In summary, for the four categories in the case study, we generated a total of 4,800 CAD data for each 2D and 3D scope. In terms of CAM data, we collected 380 for 2D and 611 for 3D scope.

Model implementation

Step 2 of the method implements deep learning models on the data generated from Step 1 for assembly quality inspection. In 2D scope, we focus on object detection tasks. The SWDA model (Saito et al. 2019) was chosen as the UDA model for the case study, as it has demonstrated superior performance on the Sim-to-Real datasets SIM 10K (Johnson-Roberson et al. 2016) and Cityscapes (Cordts et al. 2016), according to the deep domain adaptive object detection survey (Li et al. 2020). These two datasets are in similar domains as our 2D CAD data and CAM data. The SWDA model is an adversarial-based UDA model that employs the FRCNN model (Ren et al. 2015) as its base detector. As shown in Fig. 3c, it adds two discriminators, Dlocal and Dglobal, to the FRCNN model for weak global alignment and strong local alignment. Dlocal focuses on strongly aligning the local features such as texture and color, while Dglobal prioritizes aligning globally similar

images with less attention on the globally dissimilar ones (Saito et al. 2019).

The FRCNN model (Ren et al. 2015) was selected as the detector for the TL, SL, and w/o methods since it is the base detector of the SWDA model. Besides, according to the deep domain adaptive object detection survey (Li et al. 2020), it is the base detector of most domain adaptive object detection models. Moreover, it is a widely used baseline model in computer vision, with promising performance on various datasets (Zaidi et al. 2022). Figure 3c summarizes all 2D models used in our study.

In 3D scope, we focus on classification tasks. The PointDAN model (Qin et al. 2019) was chosen as the UDA model for the case study due to its promising performance on the Sim-to-Real datasets ModelNet (Wu et al. 2015) and ScanNet (Dai et al. 2017). These two datasets are in similar domains as our 3D CAD data and CAM data. The PointDAN model is an adversarial-based UDA model designed for point cloud data. As shown in Fig. 4g, it jointly aligns the global and local features. It uses a maximum classifier discrepancy (MCD) network (Saito et al. 2018) for global feature alignment and a self-adaptive (SA) node module with a node attention component for local feature alignment (Qin et al. 2019).

The PointNet model (Qi et al. 2017a) is used as a backbone feature extractor of PointDAN. A previous study (Zhu et al. 2021) has evaluated the performance of PointNet, PointNet++ (Qi et al. 2017b), PointDAN, and a self-supervised learning domain adaptation point cloud model (Achituve et al. 2021) for automatic assembly quality inspection. Given the superior performance of PointDAN and PointNet++, they were selected for this study. Specifically, we chose PointNet++ as the classifier for the TL, SL, and w/o methods. It is an advanced and extended version of the PointNet model and a state-of-the-art baseline model for point clouds classification (Guo et al. 2020). It works directly with point clouds by focusing on their global and local geometric features. All 3D models used in our study are summarized in Fig. 4g.

Experiments

The selected models and the number of data samples used in the experiments are summarized in Tables 1 and 2. Table 1 provides a summary of the 2D experiments, while Table 2 provides a summary of the 3D experiments.

In the experiments, we used 180 real-world samples for comprehensive testing in both 2D and 3D scopes. The dataset was split into training and validation sets at an 80:20 ratio. For the UDA and TL experiments, we reused the CAM data from the SL experiments. Specifically, in the TL experiments, we adopted a few-shot transfer learning strategy by fine-tuning the model with a limited amount of CAM data. We used the models trained in the w/o experiment and fine-tuned them

with 160 and 20 CAM data, respectively, to assess the impact of data quantity on fine-tuning. In the TL-20 experiments, we randomly selected 25 samples from the CAM dataset, allocating 20 for training and 5 for validation, ensuring five images from each category for balanced representation. Further, since the 2D experiments focused on object detection, to evaluate the model performance on different backgrounds, the test images contained 65 images with the same background and 115 images with different backgrounds from the training images.

For all the models, we employed the architectures as outlined in their respective original repositories and publications: FRCNN,¹ SWDA,² PointNet++,³ and PointDAN.⁴ Specifically, all models in 2D scope used a Resnet-101 network pre-trained on ImageNet as the classifier. To enhance data generalization, we incorporated horizontal flips on all the training data besides the domain randomization techniques applied in data generation.

All 2D experiments were conducted using an Nvidia RTX 2080 GPU and used Stochastic Gradient Descent (SGD) as an optimizer, while all 3D experiments were run with an Nvidia Quadro P5000 GPU and used Adam as an optimizer. The hyperparameters for these experiments, detailed in Table 3, were chosen based on preliminary analysis. This analysis aimed to optimize the balance between model performance, which was reflected in both training and validation accuracy and loss metrics, and computational efficiency. Specifically, the learning rate was tested from 1×10^{-5} to 0.1, and batch sizes ranged from 1 to 64, considering the limitations of our computational resources and the complexity of the models. We tested epochs up to 300, employing an early stopping mechanism based on validation loss to prevent overfitting and ensure improvements. The parameters were optimized using a random search method for an efficient and robust exploration of the hyperparameter space. Further details about each model, including loss functions, are available in their respective original repositories and publications.

Experimental results and discussion

The evaluation results of the 2D and 3D experiments are summarized in Table 4. All experiments were repeated three times, and the average top 1 results were reported. The object detection results of the 2D experiments were measured using mean average precision (mAP) score and accuracy (Zhang and Su 2012), while the classification results of the 3D

¹ <https://github.com/jwyang/faster-rcnn.pytorch>.

² https://github.com/VisionLearningGroup/DA_Detection.

³ https://github.com/yanx27/Pointnet_Pointnet2_pytorch.

⁴ <https://github.com/canqin001/PointDAN>.

Table 1 The models and number of data (RGB images) used in 2D experiments

Method Model	SL FRCNN	w/o FRCNN	UDA SWDA		TL-160 FRCNN		TL-20 FRCNN	
Dataset	CAM data	CAD data	CAD data	CAM data	CAD data	CAM data	CAD data	CAM data
Train	160	3840	3840	160	3840	160	3840	20
Validation	40	960	960	40	960	40	960	5
Test	All models tested on the same 180 CAM data							

Table 2 The models and number of data (point cloud) in 3D experiments

Method Model	SL PointNet++	w/o PointNet++	UDA PointDAN		TL-160 PointNet++		TL-20 PointNet++	
Dataset	CAM data	CAD data	CAD data	CAM data	CAD data	CAM data	CAD data	CAM data
Train	345	3840	3840	345	3840	160	3840	20
Validation	86	960	960	86	960	40	960	5
Test	All models tested on the same 180 CAM data							

Table 3 Hyperparameters for 2D and 3D Experiments

Experiment	2D SL	2D w/o	2D UDA	2D TL	All 3D Experiments
Model	FRCNN	FRCNN	SWDA	FRCNN	PointNet++ and PointDAN
Epochs	30	5	3	5	150
Learning Rate	0.001	0.001	0.001	0.001	0.0001
Batch Size	4	4	1	4	64

experiments were measured with accuracy. The mAP is a widely-used metric for evaluating object detection models. It evaluates both the intersection over union (IoU) precision with bounding boxes and classification accuracy. Classification accuracy, which is defined as the number of correct predictions divided by the total number of predictions, is a metric that can straightforwardly evaluate the effectiveness of the model in correctly classifying the categories. In the context of assembly quality inspection, accuracy is the crucial metric that determines the assembly quality, rather than IoU precision. Consequently, our evaluation places greater emphasis on classification accuracy.

In addition, to further validate the effectiveness of the best method, we analyzed the class-wise results of the best model with Precision, Recall, and F1 score (Zhang and Su 2012).

Results in 2D scope

Table 3 shows that the TL method attained the highest performance. By fine-tuning with only 20 CAM data, i.e., 5 CAM data per class, it achieved an accuracy of 95%. The w/o method that was only trained with CAD data has achieved 77.8% accuracy, which is higher than the SL and the same as the UDA methods. However, the SL and UDA methods exhibit different performances on the CAM data with varying backgrounds. This variation in the background of the

assembly line is typical across different factories. In the case study, we collected the 2D test CAM data with five varied backgrounds, as presented in Fig. 5. Test samples with Backgrounds 1 and 2 have the same backgrounds as the training set, while test samples with Backgrounds 3, 4, and 5 exhibit different backgrounds.

To further analyse the 2D results, Table 5 summarizes the results obtained from the 2D experiment on test sets with the same and different backgrounds compared with the training set.

Table 5 demonstrates that while the SL and UDA methods performed well on test images with the same background as the training images, their performance decreased significantly when tested on images with different backgrounds. The SL method achieved 100% accuracy on the test set with the same backgrounds as the training set, but this accuracy dropped to 59.1% when tested on data with different backgrounds. Similarly, the accuracy of the UDA method dropped from 87.7% to 72.2% under the same test conditions.

In contrast, the TL method showed promising performance in both settings. The TL-20 experiment, which fine-tuned the w/o model with 20 CAM data, i.e., 5 CAM data per category, has achieved an accuracy of around 95% while testing on all backgrounds. It increased the accuracy of the w/o method by 17.2%. When we increased the num-

Table 4 Results of the 2D and 3D experiments on the test set of CAM data

Method	Results of the 2D experiments (Test on 180 2D CAM data)		Results of the 3D experiments (Test on 180 3D CAM data)
	mAP (%)	Accuracy (%)	Accuracy (%)
SL	76.7	73.9	71.2
w/o	67.3	77.8	77.3
UDA	82.8	77.8	83.5
TL-160	96.6	96.7	80.2
TL-20	93.0	95.0	77.8

The best results marked in bold

Fig. 5 Samples of test images with different backgrounds. Backgrounds 1 and 2 match training images, while Backgrounds 3, 4, and 5 differ from training images



(a) Background 1 (b) Background 2 (c) Background 3 (d) Background 4 (e) Background 5

Table 5 Results of the 2D experiments on the test sets of CAM data with different backgrounds

Method	Test on 65 CAM data with the same backgrounds as training		Test on 115 CAM data with different backgrounds from training	
	mAP (%)	Accuracy (%)	mAP (%)	Accuracy (%)
SL	100.0	100.0	64.9	59.1
w/o	66.7	69.2	69.8	82.6
UDA	92.8	87.7	79.7	72.2
TL-160	100.0	100.0	96.0	94.8
TL-20	96.6	95.4	93.5	94.8

The best results marked in bold

ber of fine-tuning images to 160, the TL-160 increased the accuracy of TL-20 by 1.7%, which was not significant.

Furthermore, the w/o method, trained only on CAD data, exhibits inconsistent performance when tested on CAM data with different backgrounds. We assessed its performance on each of the five backgrounds individually and found that it achieved classification accuracies of 42.3%, 87.2%, 88.6%, 83.8%, and 76.7%, respectively. Among these backgrounds, it achieved the lowest performance in Background 1.

Discussion on 2D scope

As shown in Table 5, the accuracy of the SL method decreased by 41.9% when tested on images with different backgrounds than those in the training data, indicating overfitting of the model to its training environment. This behavior may be attributed to the limited variability in the small amount of training data, resulting in a narrow model that cannot generalize well to new conditions.

In terms of the w/o method, it obtained an accuracy of over 75% on Backgrounds two to four. This result suggests

that the 2D CAD data generated in this study with domain randomization may be a reasonably good representation of the categories in the case study. Thus, a model trained only with it may exhibit promising performance. However, the w/o method performed poorly, with an accuracy of less than 50%, on Background 1. One possible explanation of this discrepancy is that the white background color present in Background 1 resulted in unbalanced image contrast, making it challenging for the model, which was trained on CAD data with random backgrounds, to capture the semantic features of the CAM data.

The TL method demonstrated the best performance in the 2D experiments. By fine-tuning the w/o model with only a few CAM data, the TL method achieved the highest accuracy across all backgrounds. This superior performance may be attributed to its training approach, which incorporates both CAD and CAM data. Utilizing CAD data domain randomization techniques allows the method to gain generalized knowledge of high-level global features. Simultaneously, the fine-tuning with CAM data offers specific insights into the real domain. This combination of generalized and spe-

cific learning results in enhanced performance across various backgrounds. Using only five images per category for fine-tuning also indicates the possibility of the FRCNN model in few-shot transfer learning.

Despite also being trained on both CAD data and CAM data, the UDA method exhibited overfitting to its training backgrounds. A possible reason may be the unbalanced amount of training data in the source and target domains, i.e., 3840 CAD data and 160 CAM data. Given that the SWDA model employs an adversarial-based domain adaptation strategy, it necessitates training the discriminator with images from both domains. Consequently, an unbalanced dataset could potentially result in overfitting issues. With limited CAM data, the UDA model may have over-relied on the background features present in CAM data. Therefore, it improved the performance of the w/o method only on test data with the same background as the training data.

Overall, the TL method with the FRCNN model exhibited the highest performance across all the 2D experiments. To further validate its effectiveness, we analysed its class-wise results. Table 6 presents a summarized detailed classification report on the test set.

Table 6 illustrates that the TL-20 experiments have attained promising results with over 90% F1 score in all categories of the case study.

In summary, the TL method has demonstrated superior performance on test images with various backgrounds, indicating that its generalization ability is not impacted by the backgrounds of the test data. As such, the TL method could be trained once and utilized in various factories with diverse manufacturing environments. Furthermore, by requiring only five annotated CAM data per category for training, the TL method managed to save the effort of manual data collection and annotation in the manufacturing industry. Therefore, it is the recommended method for utilizing synthetic data in 2D scopes.

Results and discussion on 3D scope

Table 3 presents the results of 3D experiments, showing that the UDA method attained the highest accuracy of 83.5%. However, the differences in the results among all experiments were not significant.

The SL method in 3D scope recorded the lowest accuracy of 71.2%, which was unexpected since the 3D models should not be affected by the backgrounds as the 2D models. This could be due to the limited number of training samples in CAM data, i.e., 345 CAM data.

On the other hand, the w/o method, trained only with CAD data, reached an accuracy of 77.3%, which is higher than the result of the SL method. This indicates that the model only trained with CAD data may be able to capture good semantic features representing the categories in the case study. Thus,

the 3D CAD data generated by this paper has a relatively narrow domain gap from the 3D CAM data. Consequently, the UDA and TL methods did not improve the performance of the w/o method significantly by adding CAM data to the training.

2D and 3D comparison

A comparison of the accuracies between the 2D and 3D experiments presented in Table 4 reveals that the 2D experiments achieved higher accuracy than the 3D experiments in all methods except for the UDA method. The poorer performance of the 2D UDA methods might be attributed to the backgrounds of the CAM data, which impacted the performance of the 2D experiments but not the 3D ones. Thus, despite the various backgrounds, the 2D experiments have demonstrated better performance than all the 3D experiments. This may have two reasons:

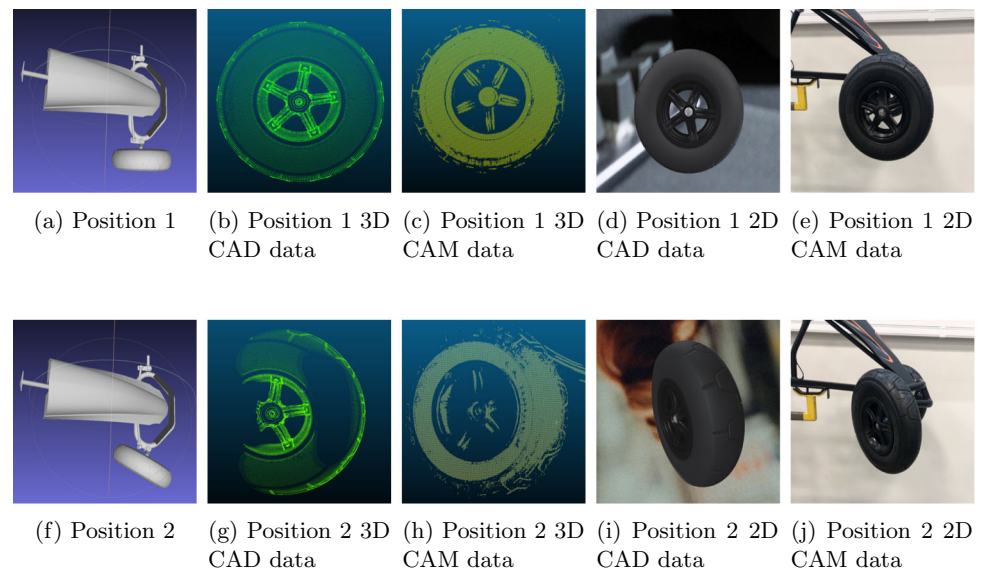
1) The backbones used in the 2D and 3D models were different. All the 2D models were trained with backbone classifiers pre-trained on ImageNet, while the 3D models were trained from scratch. Given that point cloud data are, in general, more challenging to gather than image data, it is more difficult to obtain a robust pre-trained backbone on point cloud data than on image data.

2) The data quality may vary between the 2D and 3D CAM data. During the data collection, the rotations of the steering shaft led to different end positions of the front wheel, resulting in varying CAM data quality. Figure 6 presents some samples of the 2D and 3D data when the front wheel is in different end positions.

In Fig. 6, Position 1 represents the wheel being parallel to both the 2D and 3D cameras, allowing the cameras to capture all the relevant information about the wheel. As shown in Fig. 6a to e, all data in Position 1 are of good quality. However, in Position 2, the steering shaft is rotated to its maximum angle, resulting in a 60-degree angle between the wheel and the camera. As illustrated in Fig. 6h, the 3D CAM data are of low quality in this position due to the limited coverage of the laser scan. A comparison of Fig. 6g and h reveals that the 3D CAM data captures less information than the 3D CAD data in Position 2. While both datasets exhibit some missing information, the key features for categorizing the four classes in this case study, i.e., the screw and wheel rim, are less captured in the 3D CAM data, posing challenges for model classification. Conversely, Fig. 6i and j show that the 2D CAM data contains similar information to that of the 2D CAD data, indicating it has less noise and better data quality than the 3D CAM data. Most of the incorrect predictions in all 3D experiments occurred on the low-quality 3D CAM data. The missing information on the rim and screw may prevent the PointNet++ and PointDAN models from identifying

Table 6 Classification report of the TL-20 experiments on the full test set with 180 CAM data

	Precision (%)	Recall (%)	F1 score (%)	Support
Class 1	100	100	100	45
Class 2	100	95	98	44
Class 3	87	100	93	45
Class 4	95	85	90	46
Average	95	95	95	180 (Total)

Fig. 6 Samples of the 2D and 3D CAD data and CAM data when the wheel is in different end positions

important features, leading to a misclassification of the point clouds.

In addition, while comparing the loss-developed curves of the 2D and 3D experiments, we observed consistent behavior across repeated experiments. All models have been trained until convergence. The UDA model experienced slower convergence compared to the SL and TL models, which could be attributed to the complexity of simultaneously training on data from different domains. Besides, although all models eventually reached a stable state, the curves in the 3D experiments continued to exhibit some fluctuations towards the end, diverging from the 2D experiments, which completely flattened out. This distinction could be potentially caused by the noise in the 3D CAM data.

In summary, the 2D experiments achieved higher accuracy than the 3D experiments. Among the 2D models, the TL method exhibited the best performance.

In addition to the model performance, the hardware requirements of the 2D methods are more cost-effective and user-friendly than those of the 3D methods. Generating 3D CAM data often requires costly high-resolution cameras and additional fixtures to scan the camera, whereas 2D data generation can be accomplished with low-cost webcams. Additionally, the issue of low-quality point clouds and missing information from 3D cameras, as encountered in our case

study, is likely to recur in scenarios where objects with different assembly end positions are captured by a fixed-path camera. Addressing this challenge might necessitate the use of multiple cameras or more adaptable fixtures, potentially leading to increased costs. In contrast, 2D cameras generally face fewer such issues.

In conclusion, based on the evaluation results, we recommend the proposed method in 2D scope with the TL model as the best possible solution among the examined methods for automatic assembly quality inspection with synthetic data.

Conclusion

This paper presented a method for automatic assembly quality inspection using synthetic data generated from CAD models. It included two steps: automatic data generation and model implementation. To identify a well-performing method, we explored synthetic data in both 2D (images) and 3D (point clouds) formats and applied cutting-edge transfer learning and unsupervised domain adaptation models to the data. These models were compared to traditional supervised learning models in a case study focused on the quality inspection of pedal car front-wheel assembly. The key finding is that the method in 2D scope with transfer learning

using the Faster RCNN model attained the best performance with over 95% accuracy, indicating its high effectiveness for assembly quality inspection tasks.

Building on this promising result, the 2D method with transfer learning may be recommended for similar quality inspection scenarios. One strength of this method is its ability to obtain promising performance using only a few annotated real data, i.e., five real images per category, making it time-saving and cost-efficient for implementation in the industry. In addition, the evaluation results indicate that the method performance remains consistent despite the variations in assembly background, suggesting that the model could be trained once and applied across different assembly factory environments.

While this research provides initial insight into the use of synthetic data for assembly quality inspection, it is limited to a single case study and specific deep learning models. To further assess the generalizability of the method, future research will focus on creating a comprehensive benchmark dataset containing multiple industry use cases for evaluation. Additionally, considering that the models we employed were developed before 2019, we intend to explore more recent architectural advancements, such as Vision Transformer, to enhance our performance comparison.

Furthermore, despite our method showing potential in reducing manual data collection and annotation efforts, it still relies on CAM data for fine-tuning. Our next goal is to develop a model that can be trained only on CAD data. With this model, the data collection and annotation process can be avoided entirely, making it possible to design the quality inspection station before the manufacturing line is built.

Acknowledgements We gratefully acknowledge our colleagues at the Academy and Smart Factory Lab Departments at Scania CV AB for providing the CAD models and use cases. We extend our thanks to Pooja Rangarajan and her colleagues at the Volkswagen Virtual Engineering Lab for assisting in generating synthetic images from CAD models. In addition, we express our appreciation to Juan Luis Jiménez Sánchez and Himaja Manamasa from Scania CV AB for their support in the 3D experiments.

Author Contributions All authors contributed to the study conception. Material preparation, data collection, and analysis were performed by Xiaomeng Zhu. The first draft of the manuscript was written by Xiaomeng Zhu, and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding Open access funding provided by Royal Institute of Technology. This work is partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

Data availability The data that support the findings of this study are available from the corresponding author upon request.

Declarations

Conflict of interest The authors declare no Conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Achituve, I., Maron, H., & Chechik, G. (2021). Self-supervised learning for domain adaptation on point clouds. In *Proceedings of the IEEE/CVF Winter conference on applications of computer vision*, pp. 123–133. <https://doi.org/10.48550/arXiv.2003.12641>
- Ahmad, H. M., & Rahimi, A. (2022). Deep learning methods for object detection in smart manufacturing: A survey. *Journal of Manufacturing Systems*, 64, 181–196. <https://doi.org/10.1016/j.jmsy.2022.06.011>
- Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). Yolov4: Optimal speed and accuracy of object detection. [arXiv:2004.10934](https://arxiv.org/abs/2004.10934). <https://doi.org/10.48550/arXiv.2004.10934>
- Brolin, A., Thorvald, P., & Case, K. (2017). Experimental study of cognitive aspects affecting human performance in manual assembly. *Production & Manufacturing Research*, 5(1), 141–163.
- Chou, P.-Y., Lin, C.-H., & Kao, W.-C. (2022). A novel plug-in module for fine-grained visual classification. [arXiv:2202.03822](https://arxiv.org/abs/2202.03822). <https://doi.org/10.48550/arXiv.2202.03822>
- Cohen, J., Crispim-Junior, C. F., Grange-Faivre, C., & Tougne, L. (2020). CAD-based learning for egocentric object detection in industrial context. In *15th international conference on computer vision theory and applications*, Vol. 5, pp. 644–651. <https://doi.org/10.5220/0008975506440651>
- Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., & Schiele, B. (2016). The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 3213–3223. <https://doi.org/10.1109/CVPR.2016.350>
- Dai, A., Chang, A. X., Savva, M., Halber, M., Funkhouser, T., & Nießner, M. (2017). ScanNet: Richly-annotated 3D reconstructions of indoor scenes. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 5828–5839. <https://doi.org/10.48550/arXiv.1702.04405>
- Dastoorian, R., Elhabashy, A. E., Tian, W., Wells, L. J., & Camelio, J. A. (2018). Automated surface inspection using 3D point cloud data in manufacturing: A case study. In *International Manufacturing Science and Engineering Conference*, Vol. 51371, pp. V003T02A036. American Society of Mechanical Engineers. <https://doi.org/10.1115/MSEC2018-6542>
- de Melo, C. M., Torralba, A., Guibas, L., DiCarlo, J., Chellappa, R., & Hodgins, J. (2021). Next-generation deep learning based on sim-

- ulators and synthetic data. *Trends in Cognitive Sciences*. <https://doi.org/10.1016/j.tics.2021.11.008>
- de Vin, L. J., Jacobsson, L., & Odhe, J. (2018). Game-based lean production training of university students and industrial employees. *Procedia Manufacturing*, 25, 578–585. <https://doi.org/10.1016/j.promfg.2018.06.098>
- Dekhtiar, J., Durupt, A., Bricogne, M., Eynard, B., Rowson, H., & Kiritsis, D. (2018). Deep learning for big data applications in CAD and PLM-research review, opportunities and case study. *Computers in Industry*, 100, 227–243. <https://doi.org/10.1016/j.compind.2018.04.005>
- Everingham, M., Van Gool, L., Williams, C. K., Winn, J., & Zisserman, A. (2010). The pascal visual object classes (VOC) challenge. *International Journal of Computer Vision*, 88, 303–338. <https://doi.org/10.1007/s11263-009-0275-4>
- Eversberg, L., & Lambrecht, J. (2021). Generating images with physics-based rendering for an industrial object detection task: Realism versus domain randomization. *Sensors*, 21(23), 7901. <https://doi.org/10.3390/s21237901>
- Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., Marchand, M., & Lempitsky, V. (2016). Domain-adversarial training of neural networks. *The Journal of Machine Learning Research*, 17(1), 2096–2130. <https://doi.org/10.48550/arXiv.1505.07818>
- Guo, Y., Wang, H., Hu, Q., Liu, H., Liu, L., & Bennamoun, M. (2020). Deep learning for 3D point clouds: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(12), 4338–4364. <https://doi.org/10.48550/arXiv.1912.12033>
- Haas, J. K. (2014). A history of the unity game engine. *Diss. Worcester Polytechnic Institute*, 483(2014), 484.
- Horváth, D., Erdős, G., Istenes, Z., Horváth, T., & Földi, S. (2022). Object detection using sim2real domain randomization for robotic applications. *IEEE Transactions on Robotics*. <https://doi.org/10.48550/arXiv.2208.04171>
- Huang, W., & Kovacevic, R. (2011). A laser-based vision system for weld quality inspection. *Sensors*, 11(1), 506–521. <https://doi.org/10.3390/s110100506>
- Jiao, L., Zhang, F., Liu, F., Yang, S., Li, L., Feng, Z., & Qu, R. (2019). A survey of deep learning-based object detection. *IEEE Access*, 7, 128837–128868. <https://doi.org/10.1109/ACCESS.2019.2939201>
- Johnson-Roberson, M., Barto, C., Mehta, R., Sridhar, S. N., Rosaen, K., & Vasudevan, R. (2016). Driving in the matrix: Can virtual worlds replace human-generated annotations for real world tasks? [arXiv:1610.01983v2](https://arxiv.org/abs/1610.01983v2) [cs.CV]. <https://doi.org/10.48550/arXiv.1610.01983>
- Krauß, J., Pacheco, B. M., Zang, H. M., & Schmitt, R. H. (2020). Automated machine learning for predictive quality in production. *Procedia CIRP*, 93, 443–448. <https://doi.org/10.1016/j.procir.2020.04.039>
- Krüger, J., Lehr, J., Schlueter, M., & Bischoff, N. (2019). Deep learning for part identification based on inherent features. *CIRP Annals*, 68(1), 9–12. <https://doi.org/10.1016/j.cirp.2019.04.095>
- Leu, M. C., ElMaraghy, H. A., Nee, A. Y., Ong, S. K., Lanzetta, M., Putz, M., Zhu, W., & Bernard, A. (2013). CAD model based virtual assembly simulation, planning and training. *CIRP Annals*, 62(2), 799–822. <https://doi.org/10.1016/j.cirp.2013.05.005>
- Li, W., Li, F., Luo, Y., Wang, P., et al. (2020). Deep domain adaptive object detection: a survey. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 1808–1813. IEEE. <https://doi.org/10.48550/arXiv.2002.06797>
- Li, X., Zhang, W., Ding, Q., & Sun, J.-Q. (2020). Intelligent rotating machinery fault diagnosis based on deep learning using data augmentation. *Journal of Intelligent Manufacturing*, 31, 433–452.
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., & Berg, A. C. (2016). Ssd: Single shot multibox detector. In *Computer vision—ECCV 2016: 14th European conference, Amsterdam, The Netherlands, October 11–14, 2016, proceedings, Part I 14*, pp. 21–37. Springer. https://doi.org/10.1007/978-3-319-46448-0_2
- Liu, W., Sun, J., Li, W., Hu, T., & Wang, P. (2019). Deep learning on point clouds and its application: A survey. *Sensors*, 19(19), 4188. <https://doi.org/10.3390/s19194188>
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 10012–10022. <https://doi.org/10.48550/arXiv.2103.14030>
- Maschler, B., Kamm, S., Jazdi, N., & Weyrich, M. (2020). Distributed cooperative deep transfer learning for industrial image recognition. *Procedia CIRP*, 93, 437–442. <https://doi.org/10.1016/j.procir.2020.03.056>
- Mazetto, M., Teixeira, M., Rodrigues, É. O., & Casanova, D. (2020). Deep learning models for visual inspection on automotive assembling line. *International Journal of Advanced Engineering Research and Science*, 7(4), 473–494. <https://doi.org/10.22161/ijaers.74.56>
- Michalos, G., Makris, S., Papakostas, N., Mourtzis, D., & Chrysosouris, G. (2010). Automotive assembly technologies review: Challenges and outlook for a flexible and adaptive approach. *CIRP Journal of Manufacturing Science and Technology*, 2(2), 81–91. <https://doi.org/10.1016/j.cirpj.2009.12.001>
- Miller, T. (2019). Simplified neural unsupervised domain adaptation. In *Proceedings of the conference, Association for Computational Linguistics, North American Chapter meeting*, Vol. 2019, pp. 414. NIH Public Access. <https://doi.org/10.48550/arXiv.1905.09153>
- Munappy, A. R., Bosch, J., Olsson, H. H., Arpteg, A., & Brinne, B. (2022). Data management for production quality deep learning models: Challenges and solutions. *Journal of Systems and Software*, 191, 111359. <https://doi.org/10.1016/j.jss.2022.111359>
- Nguyen, N.-D., Do, T., Ngo, T. D., & Le, D.-D. (2020). An evaluation of deep learning methods for small object detection. *Journal of Electrical and Computer Engineering*, 2020, 1–18. <https://doi.org/10.1155/2020/3189691>
- Nikolenko, S. I. (2021). *Synthetic data for deep learning* (Vol. 174). Springer.
- Oesterreich, T. D., & Teuteberg, F. (2016). Understanding the implications of digitisation and automation in the context of industry 4.0: A triangulation approach and elements of a research agenda for the construction industry. *Computers in Industry*, 83, 121–139. <https://doi.org/10.1016/j.compind.2016.09.006>
- Qi, C. R., Su, H., Mo, K., & Guibas, L. J. (2017a). Pointnet: Deep learning on point sets for 3D classification and segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 652–660. <https://doi.org/10.48550/arXiv.1612.00593>
- Qi, C. R., Yi, L., Su, H., & Guibas, L. J. (2017). Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *Advances in Neural Information Processing Systems*. <https://doi.org/10.48550/arXiv.1706.02413>
- Qiao, S., Chen, L.-C., & Yuille, A. (2021). Detectors: Detecting objects with recursive feature pyramid and switchable atrous convolution. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 10213–10224. <https://doi.org/10.48550/arXiv.2006.02334>
- Qin, C., You, H., Wang, L., Kuo, C.-C.J., & Fu, Y. (2019). Pointdan: A multi-scale 3D domain adaption network for point cloud representation. *Advances in Neural Information Processing Systems*. <https://doi.org/10.48550/arXiv.1911.02744>
- Rangarajan, P., Gupta, N., Andreas, M., Breitenfeld, A., Schulz, S., Ling, S., Kammerlocher, T., & Baier, F. (2022, December). Computer-implemented method and system for generating a syn-

- thetic training data set for training a machine learning computer vision model. European patent application, 21179758.4
- Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*. <https://doi.org/10.48550/arXiv.1506.01497>
- Saito, K., Ushiku, Y., Harada, T., & Saenko, K. (2019). Strong-weak distribution alignment for adaptive object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 6956–6965. <https://doi.org/10.48550/arXiv.1812.04798>
- Saito, K., Watanabe, K., Ushiku, Y., & Harada, T. (2018). Maximum classifier discrepancy for unsupervised domain adaptation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 3723–3732. <https://doi.org/10.48550/arXiv.1712.02560>
- Samir, K., Khabbazi, M. R., Maffei, A., & Onori, M. A. (2018). Key performance indicators in cyber-physical production systems. *Procedia CIRP*, 72, 498–502. <https://doi.org/10.1016/j.procir.2018.03.036>
- Seib, V., Lange, B., & Wirtz, S. (2020). Mixing real and synthetic data to enhance neural network training—a review of current approaches. *arXiv:2007.08781v1* [cs.CV]. <https://doi.org/10.48550/arXiv.2007.08781>
- Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30. IEEE. <https://doi.org/10.48550/arXiv.1703.06907>
- Tovey, M. (1989). Drawing and CAD in industrial design. *Design Studies*, 10(1), 24–39. [https://doi.org/10.1016/0142-694X\(89\)90022-7](https://doi.org/10.1016/0142-694X(89)90022-7)
- Tsirikoglou, A., Eilertsen, G., & Unger, J. (2020). A survey of image synthesis methods for visual machine learning. In *Computer graphics forum*, Vol. 39, pp. 426–451. Wiley Online Library. <https://doi.org/10.1111/cgf.14047>
- Unsplash. (2020). Unsplash dataset. Retrieved 01 Jun 2020, from <https://unsplash.com/data>
- Wang, J., Ma, Y., Zhang, L., Gao, R. X., & Wu, D. (2018). Deep learning for smart manufacturing: Methods and applications. *Journal of Manufacturing Systems*, 48, 144–156. <https://doi.org/10.1016/j.jmsy.2018.01.003>
- Wang, Y., Yao, Q., Kwok, J. T., & Ni, L. M. (2020). Generalizing from a few examples: A survey on few-shot learning. *ACM Computing Surveys (CSUR)*, 53(3), 1–34. <https://doi.org/10.48550/arXiv.1904.05046>
- Wong, M. Z., Kunii, K., Baylis, M., Ong, W. H., Kroupa, P., & Koller, S. (2019). Synthetic dataset generation for object-to-model deep learning in industrial applications. *PeerJ Computer Science*, 5, e222. <https://doi.org/10.7717/peerj-cs.222>
- Wu, Z., Song, S., Khosla, A., Yu, F., Zhang, L., Tang, X., & Xiao, J. (2015). 3D shapenets: A deep representation for volumetric shapes. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 1912–1920. <https://doi.org/10.48550/arXiv.1406.5670>
- Zaidi, S. S. A., Ansari, M. S., Aslam, A., Kanwal, N., Asghar, M., & Lee, B. (2022). A survey of modern deep learning based object detection models. *Digital Signal Processing*. <https://doi.org/10.48550/arXiv.2104.11892>
- Zhang, C., Xi, N., & Shi, Q. (2008). Object-orientated registration method for surface inspection of automotive windshields. In *2008 IEEE/RSJ international conference on intelligent robots and systems*, pp. 3553–3558. IEEE. <https://doi.org/10.1109/IROS.2008.4650877>
- Zhang, P. & Su, W. (2012). Statistical inference on recall, precision and average precision under random selection. In *2012 9th international conference on fuzzy systems and knowledge discovery*, pp. 1348–1352. IEEE. <https://doi.org/10.1109/FSKD.2012.6234049>
- Zhu, X., Bilal, T., Mårtensson, P., Hanson, L., Björkman, M., & Maki, A. (2023). June. Towards sim-to-real industrial parts classification with synthetic dataset. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, pp. 4453–4462. <https://doi.org/10.1109/CVPRW59228.2023.00468>
- Zhu, X., Maki, A., & Hanson, L. (2022). Unsupervised domain adaptive object detection for assembly quality inspection. *Procedia CIRP*, 112, 477–482. <https://doi.org/10.1016/j.procir.2022.09.038>
- Zhu, X., Manamasa, H., Sánchez, J. L. J., Maki, A., & Hanson, L. (2021). Automatic assembly quality inspection based on an unsupervised point cloud domain adaptation model. *Procedia CIRP*, 104, 1801–1806. <https://doi.org/10.1016/j.procir.2021.11.304>
- Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H., & He, Q. (2020). A comprehensive survey on transfer learning. *Proceedings of the IEEE*, 109(1), 43–76. <https://doi.org/10.48550/arXiv.1911.02685>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.